Projeto Prático: Aplicação Web SPA com Angular

1. Visão Geral

Chegou o momento de consolidar e demonstrar as competências adquiridas no módulo de Fundamentos de Angular!

O objetivo deste projeto é desenvolver uma aplicação web interativa, responsiva e organizada, utilizando Angular e TypeScript, aplicando os princípios de desenvolvimento baseados em componentes e a arquitetura de uma Single Page Application (SPA).

O tema do projeto é livre, à escolha de cada formando, desde que permita demonstrar a utilização das principais funcionalidades de Angular abordadas nas sessões.

Exemplos de temas possíveis:

- Gestão de tarefas e projetos;
- Catálogo de filmes, séries ou videojogos;
- Plataforma de gestão de produtos e encomendas;
- Gestão de despesas pessoais;
- Plataforma de eventos e inscrições;
- Catálogo de livros;
- Gestão de cursos e formandos.

A aplicação deverá apresentar uma estrutura organizada, separar as responsabilidades entre componentes e serviços e permitir que o utilizador interaja com os dados através de uma interface dinâmica.

2. Requisitos Obrigatórios do Projeto

Para garantir uma avaliação completa, a aplicação deverá cumprir os seguintes requisitos.

1. Estrutura Angular e Componentização

A aplicação deverá ser desenvolvida com Angular CLI e organizada através de componentes reutilizáveis.

Requisitos:

- Criar uma aplicação Angular funcional;
- Separar as diferentes áreas da interface em componentes;

- Utilizar templates HTML e ficheiros TypeScript para organizar a apresentação e a lógica;
- Utilizar componentes de forma coerente, evitando concentrar toda a aplicação num único componente.

Exemplo: uma aplicação de gestão de tarefas poderá ter componentes para o cabeçalho, formulário de criação, lista de tarefas e cartão individual de tarefa.

2. Templates, Data Binding e Renderização Dinâmica

A interface deverá apresentar dados provenientes dos componentes e reagir às ações do utilizador.

Requisitos:

- Utilizar interpolation ({{ }}) para apresentar dados;
- Utilizar property binding ([ ]) para configurar propriedades de elementos;
- Utilizar event binding (( )) para responder a eventos;
- Utilizar two-way data binding ([( )]) quando adequado;
- Utilizar @if e @for para controlar a apresentação dos elementos;
- Utilizar @switch quando existirem diferentes estados ou categorias que justifiquem a sua utilização;
- Aplicar pipes nativos para formatar dados, como datas, números ou valores monetários.

A aplicação deverá, por exemplo, permitir pesquisar, filtrar ou alterar informação e atualizar a interface sem manipulação manual do DOM.

3. Comunicação entre Componentes

A aplicação deverá demonstrar comunicação entre componentes pai e filho.

Requisitos:

- Utilizar @Input para transmitir dados do componente pai para o componente filho;
- Utilizar @Output e EventEmitter para comunicar ações do componente filho ao componente pai;
- Criar pelo menos um componente filho reutilizável que receba dados e emita um evento.

Exemplo: um componente de cartão de produto recebe os dados do produto através de @Input e emite um evento quando o utilizador pretende adicionar esse produto ao carrinho.

4. Services, Interfaces TypeScript e Injeção de Dependências

A lógica de acesso e gestão dos dados deverá estar separada da camada de apresentação.

Requisitos:

- Criar pelo menos um Service utilizando @Injectable;
- Utilizar a Injeção de Dependências para disponibilizar o serviço aos componentes;
- Criar interfaces TypeScript para representar as principais entidades da aplicação;
- Evitar concentrar toda a lógica de negócio diretamente nos componentes.

Exemplo: uma aplicação de eventos poderá definir uma interface Evento e utilizar um EventoService para disponibilizar os dados dos eventos.

5. Consumo de Dados com HttpClient

A aplicação deverá obter dados através de uma fonte de dados externa ou simulada, utilizando a abordagem ensinada nas sessões.

Requisitos:

- Utilizar HttpClient para realizar pelo menos uma requisição HTTP;
- Efetuar a requisição através de um Service;
- Tipar a resposta com uma interface TypeScript;
- Utilizar Observable e subscribe() para tratar a resposta;
- Apresentar os dados recebidos na interface;
- Prever estados de carregamento e de erro, para que o utilizador receba feedback quando os dados estão a ser carregados ou quando ocorre uma falha;
- Obrigatória a utilização do supabase como Bass utilizando os seus serviços de autenticação e a base de dados de armazenamento;
- O Projeto deverá ter a configuração de produção no github Actions e ser servido no github pages;

- O projeto deverá conter variáveis de ambiente para proteção da Key utilizada pelo supabase

6. Navegação com Angular Router

A aplicação deverá funcionar como uma SPA, permitindo navegar entre diferentes páginas sem recarregar integralmente o navegador.

Requisitos:

- Configurar as rotas da aplicação;
- Utilizar router-outlet para apresentar o componente correspondente à rota ativa;
- Utilizar RouterLink para navegar entre páginas;
- Criar pelo menos três vistas distintas, incluindo a página inicial e uma página de listagem ou gestão de dados.

Exemplo de estrutura de navegação:

- Página inicial ou dashboard;
- Página de listagem de elementos;
- Página de criação ou edição de elementos.

As páginas deverão estar organizadas de acordo com as funcionalidades do projeto.

7. Formulários Reativos e Validação

A aplicação deverá incluir pelo menos um formulário funcional, implementado com Reactive Forms.

Requisitos:

- Utilizar FormGroup e FormControl;
- Utilizar ReactiveFormsModule;
- Aplicar validações adequadas aos campos, utilizando validadores como Validators.required, Validators.email e Validators.minLength, quando aplicáveis;
- Apresentar mensagens de erro quando os dados introduzidos forem inválidos;
- Impedir a submissão enquanto o formulário estiver inválido;
- Processar os dados após uma submissão válida.

Exemplo: um formulário para registar uma tarefa deverá exigir um título e validar os restantes campos de acordo com as regras definidas para a aplicação.

8. Ciclo de Vida dos Componentes e Organização do Código

A aplicação deverá demonstrar uma utilização adequada do ciclo de vida dos componentes e boas práticas de organização.

Requisitos:

- Utilizar OnInit para inicializar dados ou executar operações necessárias ao carregamento de um componente;
- Organizar os ficheiros por responsabilidade, evitando duplicação desnecessária de código;
- Manter a lógica de acesso a dados nos Services e a lógica de apresentação nos componentes;
- Utilizar nomes claros e consistentes para componentes, serviços, interfaces, métodos e variáveis.

Quando a aplicação utilizar recursos que necessitem de limpeza, deverá ser considerada a utilização de OnDestroy.

3. Sugestões de Projetos

Se não tiveres uma ideia definida, poderás escolher um dos seguintes cenários como inspiração.

Opção A — TaskFlow: Gestão de Tarefas

Desenvolver uma aplicação para gerir tarefas pessoais ou de uma equipa.

Funcionalidades sugeridas:

- Listar tarefas e apresentar o respetivo estado;
- Criar novas tarefas através de um formulário reativo;
- Filtrar tarefas por estado ou prioridade;
- Apresentar diferentes estados, como Pendente, Em Curso e Concluída;
- Utilizar componentes reutilizáveis para apresentar cada tarefa;
- Consultar dados através de um Service e de uma fonte HTTP;
- Navegar entre o dashboard e a página de gestão de tarefas.

Opção B — CineHub: Catálogo de Filmes e Séries

Desenvolver uma plataforma para explorar filmes e séries.

Funcionalidades sugeridas:

- Obter filmes ou séries a partir de uma API;
- Apresentar os resultados em cartões reutilizáveis;
- Pesquisar títulos e filtrar resultados por género;
- Apresentar os detalhes de um elemento selecionado;
- Utilizar um formulário para registar uma avaliação ou sugestão;
- Organizar a aplicação através de diferentes rotas.

Opção C — ShopManager: Gestão de Produtos

Desenvolver uma aplicação para consultar e gerir produtos de uma loja.

Funcionalidades sugeridas:

- Consultar produtos através de um Service;
- Apresentar nome, preço, categoria e disponibilidade;
- Filtrar produtos por categoria;
- Criar ou editar produtos através de formulários reativos;
- Apresentar o estado de disponibilidade através de renderização condicional;
- Utilizar componentes reutilizáveis para os cartões de produto;
- Organizar as páginas através do Angular Router.

Estas opções são apenas sugestões. É permitido desenvolver qualquer outro tema, desde que todos os requisitos obrigatórios sejam cumpridos.

4. Entrega e Alojamento

A entrega deverá incluir:

1. **Repositório público no GitHub**, contendo o código-fonte completo da aplicação;
2. **Aplicação publicada e funcional**, acessível através de um link público;
3. **Ficheiro README.md**, contendo:
    - Uma breve descrição do projeto;
    - As principais funcionalidades implementadas;
    - O link direto para a aplicação publicada;

- As instruções necessárias para instalar as dependências e executar o projeto localmente;
- A indicação da API ou fonte de dados utilizada;

4. **Código organizado**, com componentes, serviços, interfaces e restantes ficheiros necessários à execução do projeto.

A publicação deverá permitir aceder à aplicação e navegar entre as suas páginas. Caso seja utilizado um serviço externo, este deverá estar acessível no ambiente de demonstração.

5. Apresentação do Projeto

Durante a apresentação, cada formando deverá demonstrar a aplicação em funcionamento (GITHUBPAGES).

Data de entrega:
01/11/26 – Até as 23:59

Apresentação:
02/11/26 – **PONTUALMENTE** as 09:00.